Does the enforcement of legal play through logit masking and environment constraints accelerate agent convergence?

by

Francois Qian
URN: 6702759/7

A dissertation submitted in partial fulfilment of the
requirements for the award of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

May 2026

Department of Computer Science
University of Surrey
Guildford GU2 7XH

Supervised by: Nishanth Sastry

I declare that this dissertation is my own work and that the work of others is acknowledged and indicated by explicit references.

Francois Qian
May 2026

© Copyright Francois Qian, May 2026

Abstract

1. The convergence of self-play in a pseudo-legal environment toward strict legal play.
2. The impact of logit masking on learning dynamics, specifically regarding the "grokking" of physical board mechanics.

In the development of neural agents for combinatorial games, a fundamental tension exists between explicit policy pruning and emergent rule internalization. This thesis investigates two primary paradigms: **Logit Masking** and **Punishment Propagation**.

- **Hypothesis:** While logit masking provides superior convergence speed, unmasked training (Punishment Propagation) forces the network to "grok" the underlying physical mechanics of the environment, potentially yielding more robust latent representations.

Acknowledgements

I would like to thank my supervisor, Nishanth Sastry, for overseeing the project.

Contents

1 Introduction **14**
1.1 Project Background 14
1.2 Project Description 14
1.3 Aims and Objectives 15
1.4 Thesis Structure 15

2 Literature Review **16**
2.1 Chess as an MDP 16
2.2 Chess Played by Machines 16
2.2.1 AlphaZero and MuZero 16
2.2.2 Transformers in Chess 17
2.3 Invalid Action Masking 17
2.4 Learning Dynamics 17
2.5 Summary and Research Gap 18

3 System Design and Methodology **19**
3.1 System Architecture Overview 19
3.2 Environment Formulation and State Representation 19
3.2.1 Dual Ruleset MDPs 19
3.2.2 State Canonicalization and Tensor Formulation 19
3.2.3 Network Topology 20
3.2.4 Output Heads 20
3.2.5 Bipartite Monte Carlo Tree Search (MCTS) 20
3.2.6 Selection and PUCT Formulation 20
3.2.7 Exploration Mechanisms 21
3.2.8 Simulation Integrity and Masking 21
3.3 Reinforcement Learning Pipeline 21
3.3.1 Self-Play and Replay Buffer 21
3.4 Optimisation and Loss Function 21
3.5 Experimental Design and Evaluation Metrics 22
3.5.1 The Configuration Matrix 22
3.5.2 Quantitative Metrics 22

4 Implementation Details **23**
4.1 Software Stack and Overview 23

4.2 Chess Engine Core ... 24
4.2.1 Bitboard layout and memory footprint: ... 24
4.2.2 Compile-time mask generation: ... 24
4.2.3 Move generation with blocker isolation: ... 24
4.2.4 Stack-allocated moves: ... 24
4.2.5 Zobrist hashing: ... 24
4.3 Bipartite Monte Carlo Tree Search ... 24
4.4 Node and edge types: ... 25
4.5 Arena allocators: ... 25
4.6 Node expansion: ... 25
4.6.1 Value Propagation: ... 26
4.6.2 Promotion splitting: ... 26
4.6.3 PUCT and edge selection: ... 27
4.6.4 Mask application and renormalisation: ... 27
4.6.5 PUCT and edge selection ... 27
4.6.6 Parallel search: ... 28
4.7 Neural Network and Tensor Formulation ... 28
4.7.1 State canonicalisation: ... 28
4.7.2 Input tensor layout: ... 29
4.7.3 Two-pass forward: ... 29
4.7.4 Transformer architecture: ... 29
4.7.5 Output processing: ... 30
4.8 Training Pipeline ... 30
4.8.1 Self-play loop (play()): ... 30
4.8.2 Expand batch (expand_batch): ... 30
4.8.3 Replay buffer: ... 30
4.8.4 Self-annealing loss: ... 30
4.8.5 Optimiser and LR schedule: ... 30
4.9 Performance and Correctness ... 31
4.9.1 Correctness: ... 31
4.9.2 Profiling and throughput: ... 31
4.9.3 Scalability: ... 31
4.10 Summary ... 31

5 Results and Evaluation ... 32
5.1 Experimental Setup ... 32
5.2 Ruleset Convergence Analysis ... 32
5.3 Masking and Grokking Analysis ... 32

6 Conclusion ... 33
6.1 Project Evaluation ... 33
6.2 Limitations and Future Work ... 33

7 Statement of Ethics ... 34

Bibliography ... 35

List of Figures

List of Tables

Glossary

$Q(s, a)$	The total reward accumulated from state
$C(s)$	CPUCT
S_i	Upper confidence bound
$\overline{x_i}$	The Mean Value of the current node
c	The Exploitation parameter
N	The current node's Visit Count
n_i	The Visit Count of the child

Abbreviations

MCTS	Monte Carlo Tree Search
UCB1	Upper Confidence Bound 1
PUCT	Predictor + Upper Confidence Bound applied to Trees
CPUCT	Exploration Predictor Constant

Chapter 1

Introduction

1.1 Project Background

Modern machine learning often relies on scaling data and compute over opaque architectures. However, these models are frequently decoupled from environment transition dynamics via **Logit Masking**—a heuristic that artificially zeroes out the probability of illegal actions prior to softmax activation. While computationally efficient, this deprives the network of negative feedback for invalid predictions, bypassing the model's need to internally represent the mechanics of play. Furthermore, standard reinforcement learning (RL) agents are prevented from making invalid state transitions by hardcoded environmental constraints. This precludes the observation of how internal action space pruning influences learning before strategic evaluation develops. Chess provides an optimal substrate for this investigation: it presents highly ambiguous intermediate states bound by objective ground truths and a mathematically perfect, yet computationally intractable, solution space.

1.2 Project Description

Existing literature rarely assesses the impact of these environmental guardrails on the learning dynamics of transformer-based agents. This project investigates two orthogonal axes of environmental constraint: Action Space Constraints and Logit Masking.

To facilitate explicit measurement of model interpretability, a custom two-pass autoregressive transformer encoder and a bipartite Monte Carlo Tree Search (MCTS) are introduced. This architecture decouples piece selection from destination selection, providing a granular window into the model's spatial reasoning.

The experimental configurations are defined across two axes:

1. **Axis 1: Rule Set Convergence (Action Space Constraints)**
   - **Control:** Training on a strict legal ruleset.
   - **Test:** Training on a pseudo-legal ruleset where the terminal state is the simpler, denser king capture $\left(K_{\text{cap}}\right)$ over the highly sparse, complex $(A_l = \emptyset)$. This variant of Chess fundamentally alters the MDP reward function to $R(s, a)$ yielding a potentially smoother reward landscape.

- **Hypothesis:** The network first bootstraps learning the rules and goals of the environment. Ideal play in the pseudo-legal space A_P may converge to standard Chess where concepts such as pins and checkmates emerge as survival heuristics to prevent or secure king capture. Conversely, the removal of environmental guardrails may degrade MCTS value estimation, preventing policy bootstrapping.

2. **Axis 2: Logit Masking**
   - **Control:** Masked logits (illegal moves are mathematically eliminated prior to selection).
   - **Test:** Unmasked logits (illegal moves are permitted by the network but penalized by the environment/loss function).
   - **Hypothesis:** Unmasked training will incur an initial sample-efficiency penalty by introducing the dual-optimization problem of learning game mechanics alongside strategy, but may lead to more robust internal representations reflecting potentially stronger play long term.

1.3 Aims and Objectives

The primary aim of this project is to train multiple transformer-based agents via self-play reinforcement learning across varying rule sets and action spaces, quantitatively evaluating their performance, learning trajectories, and emergent behaviors.

Specific objectives include:

- Implement a highly optimized, dual-ruleset (legal and pseudo-legal) Chess library to facilitate high-throughput RL self-play.
- Architect and implement a two-pass autoregressive transformer and bipartite MCTS to explicitly model piece-then-square action selection.
- Train four distinct model configurations via self-play: (Legal, Masked), (Legal, Unmasked), (Pseudo-Legal, Masked), and (Pseudo-Legal, Unmasked).
- Quantitatively evaluate convergence rates, illegal move proposal frequencies, and the emergence of rule-abiding play across the unconstrained configurations.
- Correlate the emergence of legal play with the attention distributions of the two-pass encoder to quantify spatial reasoning.

1.4 Thesis Structure

The remainder of this dissertation is structured as follows:

- **Chapter 2** reviews the literature surrounding reinforcement learning, transformer architectures, and action masking.
- **Chapter 3** details the system design, including the bipartite MCTS and two-pass encoder architecture.
- **Chapter 4** outlines the implementation details of the Rust engine and training pipeline.
- **Chapter 5** presents the results and evaluation of the four experimental configurations.
- **Chapter 6** concludes the project and discusses avenues for future work.

Chapter 2

Literature Review

2.1 Chess as an MDP

Chess is a deterministic, zero-sum, perfect-information game. It is formally modeled as a Markov Decision Process (MDP) defined by a tuple (S, A, T, R), where S is the state space ($|S| \approx 10^{\{40\}}$), A is the action space, $T : S \times A \to S$ is the deterministic transition function, and $R : S \times A \to \{-1, 0, 1\}$ is the reward function (Shannon, 1950).

Standard RL implementations enforce a legal action space $A_{L(s)} \subset A$ where A_L contains only moves that comply with the rules of chess (e.g., preventing self-check) (World Chess Federation (FIDE), 2023). This project introduces a relaxed pseudo-legal action space $A_{P(s)}$ where $A_{L(s)} \subset A_{P(s)} \subset A$, permitting all geometrically valid piece movements and shifting the terminal condition from the abstract checkmate to the explicit king capture $\left(K_{\text{cap}}\right)$.

Typical agents produce a single policy distribution representing every possible piece movement on all 64 squares ($|A| = 4672$). Factoring the joint probability $P(a \mid s)$ into $P(\text{from} \mid s) \times P(\text{to} \mid s, \text{from})$ reduces output layer parameters significantly (e.g., from d_model $\times$ 4672 to d_model $\times$ 64).

For any state s, an optimal policy π^* trained in A_P must implicitly learn to prune the action space, such that:

$$\pi^*(a \mid s) \to 0 \quad \text{for all } a \in \left(A_{P(s)} \setminus A_{L(s)}\right)$$

In this paradigm, concepts such as pins and checkmates are not hardcoded environmental constraints, but emergent survival heuristics within the value landscape.

2.2 Chess Played by Machines

2.2.1 AlphaZero and MuZero

The paradigm of self-play RL in chess was defined by AlphaZero (Silver *et al.*, 2017), which combined Monte Carlo Tree Search (MCTS) with deep convolutional neural networks (CNNs) for policy and value evaluation. However, AlphaZero relies entirely on a hardcoded environment to generate $A_{L(s)}$, bypassing the need for the network to internalize the rules of the game.

Subsequent architectures, such as MuZero (Schrittwieser *et al.*, 2020), removed the need for a known transition dynamics model by learning a latent environment simulator. While MuZero demonstrates the ability to adhere to environment constraints implicitly within its hidden state transitions, its policy head still simulates within the legal action space during MCTS rollouts. The literature lacks insight into how the removal of environmental guardrails or action masking impacts the sample efficiency and representational robustness of the underlying network.

2.2.2 Transformers in Chess

Recent advancements demonstrate the efficacy of Transformer architectures in chess, shifting away from the spatial inductive biases of CNNs. (Monroe and Chalmers, 2024) demonstrated that Transformers can achieve Grandmaster-level performance without explicit search, relying entirely on attention mechanisms to evaluate board states. Furthermore, (Jenner *et al.*, 2024) provided evidence of learned look-ahead within chess-playing neural networks, showing that attention heads naturally encode future board states.

Standard chess engines flatten the action space into a single discrete distribution. Decoupling piece selection $P(\text{from} \mid s)$ and destination selection $P(\text{to} \mid s, \text{from})$ forces attention mechanisms to explicitly map geometric constraints. This factored approach provides an observable window into distinct failure modes: attention failure (identifying movable pieces) and kinematic failure (how pieces move).

2.3 Invalid Action Masking

In policy gradient algorithms, it is standard practice to prevent the selection of illegal actions via Invalid Action Masking (Logit Masking). Masking applies a transformation to the policy logits z prior to the softmax activation:

$$P(a_i \mid s) = \frac{\exp(z_i + M_i)}{\sum_j \exp\big(z_j + M_j\big)}$$

where $M_i = -\infty$ if $a_i \notin A_{L(s)}$, and 0 otherwise.

While employed to accelerate convergence, (Huang and Ontañón, 2020) demonstrated that invalid action masking significantly improves sample efficiency by collapsing the exploration space. However, masking prevents punishment from invalid actions from propagating back through the network. This is critical for MCTS algorithms, where simulations into invalid state transitions degrade search efficiency and poison the policy head.

2.4 Learning Dynamics

The requirement for an unmasked network to learn the rules of chess from scratch introduces complex learning dynamics. (Power *et al.*, 2022) identified the phenomenon of "grokking," where neural networks trained on small algorithmic datasets exhibit delayed generalization long after overfitting the training data. The phenomenon arose from the result of weight decay pressure, showcasing the importance of weight regularization. Chess is an algorithmic dataset embedded within a strategic MDP, showcasing the potential for analogous phase transitions.

(Bengio *et al.*, 2009) formalized curriculum learning, finding that machine learning models converge more effectively if training data is organized in a sequence of increasing complexity. The authors demonstrate this strategy effectively functions as a regularization method, guiding the model to superior local minima often inaccessible through data shuffling. Theoretically, a pseudo-legal environment serves as an implicit curriculum, where an agent first receives rewards through piece kinematics and king capture before developing legal survival heuristics.

(Sharma *et al.*, 2017) introduced a framework that exploits the compositional nature of discrete action spaces by factoring them into independent basis-like components. This enables cross-action learning, allowing the agent to generalize actions that share common factors.

2.5 Summary and Research Gap

While the efficacy of action masking is well-documented in general RL (Huang and Ontañón, 2020), and Transformers have proven capable of modeling chess (Jenner *et al.*, 2024), the intersection of these domains remains unexplored. Existing literature does not isolate the impact of environmental guardrails on the representational quality of Transformer-based MCTS agents.

While the efficacy of action masking is well-documented, and Transformers have proven capable of modeling chess, the intersection of these domains remains unexplored. In the context of unmasked, pseudo-legal chess training, the network faces a dual-optimization problem: it must learn the physical constraints of the board (the rules) and the strategic evaluation of states (the game). We hypothesize that the network will exhibit a grokking-like phase transition: an initial period of high illegal-move proposals, followed by a sharp convergence toward $A_{L(s)}$ as physical constraints are internalized, preceding the development of high-level strategy.

In the context of unmasked, pseudo-legal chess training, the network faces a dual-optimization problem: it must learn the physical constraints of the board (the rules) and the strategic evaluation of states (the game). It is hypothesized that the network will exhibit a grokking-like phase transition: an initial period of high illegal-move proposals (random walk), followed by a sharp convergence toward $A_{L(s)}$ as physical constraints are internalized, preceding the development of high-level strategy.

This project addresses this gap by training four distinct configurations across two orthogonal axes: Action Space Constraints (Legal vs. Pseudo-Legal) and Logit Masking (Masked vs. Unmasked). By utilizing a factored two-pass autoregressive encoder, this research provides a novel, quantitative analysis of how rule internalization and action space pruning manifest within the attention layers of a chess-playing agent.

Chapter 3

System Design and Methodology

3.1 System Architecture Overview

The system is designed as a closed-loop reinforcement learning pipeline, decoupling the environment transition dynamics from the neural network's internal representations. The architecture consists of three primary components: a custom deterministic environment simulator supporting dual rulesets, a two-pass autoregressive Transformer encoder, and a bipartite Monte Carlo Tree Search (MCTS) algorithm. The system operates via asynchronous self-play, generating trajectories that are aggregated into a global replay buffer to optimize the network via gradient descent.

3.2 Environment Formulation and State Representation

3.2.1 Dual Ruleset MDPs

1. **Legal Environment ($\boldsymbol{E_l}$)**: Adheres to standard FIDE rules. The action space ($A_{l(s)}$) is strictly contained to moves that do not leave the king in check. The terminal reward $R(s, a)$ is evaluated upon Checkmate, Stalemate or standard draw conditions.
2. **Pseudo-Legal Environment ($\boldsymbol{E_p}$)**: Relaxes the action space ($A_{p(s)}$), permitting all geometrically valid piece movements regardless of king safety. The terminal conditions is shifted to the explicit king capture (K_{cap}). FIDE draw conditions are maintained to prevent infinite rollouts and support correct transition into legal play.

3.2.2 State Canonicalization and Tensor Formulation

To enforce symmetric learning and halve the state space, all board states are canonicalized to the perspective of the side-to-move. If it is Black's turn, the board and castling rights are geometrically flipped. The state s is mapped to the spatial tensor representation $X \in \mathbb{R}^{64\times 14}$ and a scalar meta-tensor $M \in \mathbb{R}^5$.

- **Spatial Tensor**: Comprises 12 piece bitboards (6 piece types $\times$ 2 colors), 1 en-passant target plane, and 1 selected-square plane (utilized in the second pass of the network).
- **Meta Tensor**: Encodes the 4 castling rights (one-hot) and the half-move clock (scaled by $\frac{1}{100}$).

To explicitly measure spatial reasoning, the system factors the joint probability of a move $P(a|s)$ into an autoregressive sequence: $P(a|s) = P(\text{from}|s) \times P(\text{to}|s, \text{from})$. The network executes two forward passes using shared weights:

1. **Pass 1 (Piece Selection)**: The network evaluates the board state with the selected square plane zeroed out, outputting a policy distribution over the 64 squares to select the origin square (*from*). The value head will evaluate the board state.
2. **Pass 2 (Destination Selection)**: The origin square is encoded into the 14th plane of the spatial tensor. The network re-evaluates the state, outputting a policy distribution over the 64 squares to select the destination (*to*). The value head will evaluate the position given a piece to move, effectively evaluating the viability of the selected piece.

Note on Promotions: Because the factored $64 \to 64$ architecture lacks a third dimension for promotion piece selection, promotions are handled dynamically during MCTS expansion. If a pawn reaches the 8th rank, the destination edge is expanded into four distinct sub-edges (Q, R, B, N), dividing the prior probability equally.

3.2.3 Network Topology

The model uses a Transformer Encoder architecture without spatial inductive biases (e.g., convolutions). The spatial tensor X is linearly projected to d_{model}, and $2D$ learned positional embeddings are added to retain geometric context. The meta-tensor M is linearly projected and concatenated as a pseudo-[CLS] token, resulting in a sequence length of 65.

The model is scaled symmetrically:

- $n_{\text{heads}} = 8$, $n_{\text{layers}} = 8$, $d_{\text{model}} = 8 * 64$, $d_{\text{ff}} = 8 * 4 * 64$

3.2.4 Output Heads

The final latent representation is routed into two distinct heads:

1. **Policy Head**: A linear projection of the 64 spatial tokens to $\mathbb{R}^{64}$, representing unnormalised logits for square seletion.
2. **Value Head**: A linear projection of the [CLS] token to $\mathbb{R}^3$, passed through softmax to predict Win/Draw/Loss probabilities. In Pass 1, this evaluates the raw state; in Pass 2, it evaluates the state conditioned on the highlighted origin square.

3.2.5 Bipartite Monte Carlo Tree Search (MCTS)

To accomodate the two-pass autoregressive policy, a bipartite MCTS is implemented. The search tree alternates between two node types:

- **Selection Nodes (N_{select})**: Represents a board state. Edges represent selection of an origin square.
- **Action Nodes (N_{move})**: Represents a board state plus selected origin square. Edges represent the destination square, transitioning the environment to an N_{select} node.

3.2.6 Selection and PUCT Formulation

During traversals, edges are selected using a modified Predictor + Upper Confidence Bound applied to Trees (PUCT) algorithm. To mitigate draw-seeking behaviour, a contempt factor is baked into the empirical action value $Q(s, a)$.

- $Q(s, a) = W - L - 0.05D$
- $U(s, a) = c_{\text{puct}} P(s, a) \frac{\sqrt{\sum_b N(s,b)}}{1+N(s,a)}$(Exploration)

- $\text{PUCT}(s, a) = Q(s, a) + U(s, a)$

where W, D, L are the mean value estimates for the edge, $P(s, a)$ is the prior probability, and N is the visit count.

3.2.7 Exploration Mechanisms

To ensure robust exploration of the state space:

1. **Dirichlet Noise**: Applied exclusively to the root node prior probabilities: $P(s, a) = (1 - \varepsilon)P(s, a) + \varepsilon\eta$, where $\eta \sim \text{Dir}(\alpha)$ with $\alpha = 0.3$ and $\varepsilon = 0.25$.
2. **Temperature Scaling**: Move selection at the node is sampled proportionally to $N(s, a)^{\frac{1}{t}}$. The temperature t scales dynamically based on ply count and remaining material, transitioning from exploratory in the opening to deterministic in endgames.

3.2.8 Simulation Integrity and Masking

During MCTS rollouts, invalid state transitions poison the value estimations of the tree. To maintain simulation integrity, action masking is applied during the search phase across all configurations. However, in the "Unmasked" experimental configurations, the network's raw logits are not masked during the calculation of the loss function, forcing the network to internalize the penalty for assigning probability mass to illegal moves.

3.3 Reinforcement Learning Pipeline

3.3.1 Self-Play and Replay Buffer

The system generates data via highly parallelized self-play. Trajectories are stored in a First-In-First-Out (FIFO) replay buffer with a capacity of 524,288 samples. To prevent infinite games, rollouts are forcibly terminated as draws after 400 plies, or if the network's internal value estimation for a draw exceeds 0.95, scaled to 0.75 in the endgame.

3.4 Optimisation and Loss Function

The network is optimised using the AdamW optimiser with $\beta_1 = 0.9$, $\beta_2 = 0.99$ and weight decay $= 10^{-4}$. Noam learning rate scheduler is used with a factor of 0.1 and 4000 warmup steps.

For a given state s, let π be the MCTS visit count distribution, p be the network's policy prediction, z be the true game outcome, and v be the network's value prediction. The loss function is defined as: $L = (1 + \lambda)\left(-\sum_a \pi(a|s) \log p(a|s)\right) + (1 - \lambda)\left(-\sum_i z_i \log v_i\right)$

This acts as a strict mathematical curriculum. When the network proposes many illegal moves $\lambda \approx 1$, the policy loss approaches 2 and the value loss approaches 0. The network is forced to learn piece kinematics before state evaluation. As the model internalises piece kinematics, the weights balance to $1 : 1$. This approach to curriculum learning is unique to the unmasked configuration, as λ is dependent on illegal move weights (λ is always zero in masked configurations).

3.5 Experimental Design and Evaluation Metrics

3.5.1 The Configuration Matrix

To isolate the effects of environmental guardrails, four distinct models are trained from scratch using identical hyperparameters (256 batch size, 256 MCTS simulations per move). The configurations form a 2×2 matrix:

1. **Legal + Masked (Control)**: Standard AlphaZero paradigm.
2. **Legal + Unmasked**: Network must learn piece kinematics via loss penalties before strategy.
3. **Pseudo-Legal + Masked**: Network plays the K_{cap} variant, and must learn FIDE rules before strategy.
4. **Pseudo-Legal + Unmasked**: Network must learn piece kinematics and FIDE rules entirely from scratch.

3.5.2 Quantitative Metrics

The learning dynamics of each configuration are evaluated using the following metrics, logged continuously during training:

- **Average Illegal Probability**: The total probability mass assigned by the raw policy head to invalid actions, quantifying the network's internalization of the ruleset.
- **Average Game Length**: Measures in plies, indicating the transition from random walks to strategic, prolonged play.
- **Win/Draw Ratios**: Tracking the emergence of decisive outcomes versus stalemates or repetitions.
- **Value Loss Convergence**: Measures the accuracy of the network's state evaluation over time.

Chapter 4

Implementation Details

4.1 Software Stack and Overview

The system is implemented in Rust, utilising its strict aliasing rules to safely parallelise MCTS rollouts without runtime data-race checks. The neural network is implemented using the Burn deep learning framework to enable backend-agnostic tensor operations (WGPU/CUDA). The complete source code is version controlled via a Nix flake.

The main execution flow is as follows:

The Loop

```
Algorithm: Self-Play Reinforcement Learning Loop
Initialization
    Load Model weights and initialize Optimizer.
    Initialize Replay Buffer and B parallel Chess games.
Self-Play Phase (Data Generation)
    For each step in training iteration:
        MCTS Simulation:
            For N simulations:
                Traverse trees to terminal leaf nodes.
                Expand and evaluate leaves in batch using Model.
        Action Selection:
            Generate policy and value targets from tree statistics.
            Execute best move in Environment.
            Apply Dirichlet noise to root for exploration.
            If Game meets draw threshold or move limit: Terminate and reset.
        Store: Push resulting samples into Replay Buffer.
Optimization Phase (Learning)
    If Replay Buffer contains sufficient samples:
        For each gradient epoch:
            Sample batch from Buffer.
```

THE LOOP

```
        Update Model parameters via Optimizer.
        Step Learning Rate Scheduler.
    Checkpointing
    Log metrics (loss, game length, win/draw rates) to CSV.
    If iteration interval reached: Save Model snapshot to disk.
```

4.2 Chess Engine Core

4.2.1 Bitboard layout and memory footprint:

The full board state is packed primarily into 15 64-bit Unsigned integers `struct Bitboard(u64)`, representing the spatial occupancy of each piece type and colour, plus three occupancy bitboards for white, black and all. This reduces the memory footprint to 120 bytes, ensuring optimal CPU cache locality. This enables the use of hardware-level intrinsics `trailing_zeros` and `leading_zeroes` to find the LSB or MSB of a bitboard.

4.2.2 Compile-time mask generation:

Masks are precomputed drectional rays (e.g., `ROOK_ATTACKS` split into cardinal directions) evaluated at compile time using `const fn`. A between mask is also created.

4.2.3 Move generation with blocker isolation:

The engine isolates the correct directional mask to use at runtime, and uses `pop_lsb` or `pop_msb` to lookup the blocking square and get the correct `BETWEEN` mask to iterate over when adding viable squares the sliding piece can move to (Brian Kernighan's algorithm.

4.2.4 Stack-allocated moves:

Move generation utilizes stack-allocated `ArrayVec<ChessMove, 128>`. While the theoretical maximum branching factor in chess is 218, empirical average branching is ≈ 30. A 128-element capacity prevents heap fragmentation and allocator bottlenecks while safely encompassing $> 99.9\%$ of reachable states. Moves are stored in ChessPosition itself, to easily decompose into available squares to select pieces from, and available destinations given a selected piece for both select and move nodes.

4.2.5 Zobrist hashing:

To detect three-fold repetitions and cache MCTS evaluations, the engine implements Zobrist Hashing. A static table of pseudo-random 64-bit integers is generated and initialised via OnceLock. The hash of any state is the `XOR` sum of the keys corresponding to the pieces, castling rights, and en-passant files. Currently, the implementation suffers from $O(P)$ time complexity calculation from scratch where P is the number of pieces on the board, as opposed to $O(1)$ for incremental updates. This trade off was made for ease of implementation.

4.3 Bipartite Monte Carlo Tree Search

Standard MCTS implementations for chess utilize a unipartite node structure where edges represent complete moves (e.g., e2e4). To explicitly model the autoregressive two-pass network, a bipartite MCTS was engineered. The tree strictly alternates between

`PieceSelect` nodes (state s) and `PieceMove` nodes (state s + origin square u). This guarantees the search topology perfectly mirrors the factored action space, allowing the network's intermediate spatial reasoning to be cached and traversed

4.4 Node and edge types:

To handle the two-pass architecture efficiently, the network processes both `PieceSelect` and `PieceMove` inputs identically. The only distinction is the presence of a single active bit in the 14th plane (the selected square) during the second pass.

```
enum MctsNode {
    PieceSelect { data: NodeData },
    PieceMove { data: NodeData, from_sq: ChessSquare },
}
```

This bipartite structure explicitly models the two-pass autoregressive nature of the network. A `PieceSelect` node transitions via an edge (the origin square) to a `PieceMove` node, which transitions via an edge (the destination square) to the next `PieceSelect` node. This guarantees that the MCTS topology perfectly mirrors the factored action space $P(\text{from}|s) \times P(\text{to}|s, \text{from})$.

4.5 Arena allocators:

Each MCTS is initialised from a `ChessGame` with 3 pre-allocated Vecs serving as arenas for Nodes, Edges and Positions. Due to the bipartite nature of the search tree, consecutive Select and Move nodes have the same chess position, differing only in selected piece. As such, a separate position arena is used to halve memory consumption. As each game instance has their own MCTS, operations can be easily parallelised without mutexes.

4.6 Node expansion:

add leaf node… terminal state 3 fold repetition and stuff, look the pseudo code

---

Leaf Node Addition and Evaluation

Algorithm: Leaf Node Addition and Evaluation

Input: Edge index e to be expanded
Path Tracking:
  Initialize H (path hashes) with current Root position hash.
  For each node in the current selection path:
    If node is `PieceSelect`: Add position Zobrist hash to H.
Validation:
  If e already points to a child node: **Return** None.
State Transition Logic:
  Let P be the parent node of e.
  If P is a `PieceSelect` node:
    Create `PieceMove` node (storing selected square).
    Link e to the new node and **Return**.
  Else if P is a `PieceMove` node:
    State Generation:

Leaf Node Addition and Evaluation

Retrieve board from P; apply move defined by e.
Compute new Zobrist hash h_{new}.
Repetition Detection:
Count occurrences of h_{new} in global history and local path H.
If count $ge2$: Set state to `Finished(Draw)`.
Else: Set state to `check_game_state()`.
Arena Integration:
If h_{new} exists in `position_arena`: Reuse index.
Else: Push new position to arena and use new index.
Terminal Evaluation:
Create `PieceSelect` node.
If state is `Finished`:
Mark node as **Terminal**.
Map reward vector $[W, D, L]$ relative to current side-to-move.
Link e to final node and **Return**.

4.6.1 Value Propagation:

```
pub fn backprop(&mut self, value: [f32; 3], color: Color) {
    self.path.iter().for_each(|&idx| {
        let edge = &mut self.edge_arena[idx];

        let parent = &self.node_arena[edge.parent_node_idx];
        let position =
&self.position_arena[parent.get_data().chess_position_idx];
        let side_to_move = position.side_to_move;

        let value = if color != side_to_move { [value[2], value[1], value[0]] }
else { value };

        edge.total_value[0] += value[0];
        edge.total_value[1] += value[1];
        edge.total_value[2] += value[2];
        edge.visits += 1;
        edge.mean_value =
            [edge.total_value[0] / edge.visits as f32, edge.total_value[1] /
edge.visits as f32, edge.total_value[2] / edge.visits as f32];
        self.node_arena[edge.parent_node_idx].get_data_mut().visits += 1;
    });
}
```

4.6.2 Promotion splitting:

The engine detects when a promotion occurs, and creates 4 edges dividing the prior equally between them, before creating the leaf nodes and positions with promotion piece. The network considers promotion as a single move, rather than 4 distinct moves.

Bipartite MCTS Expansion and State Transition

Algorithm: Promotion handling

```
If node == PieceMove
    If side_to_move is Black; flip square
    If move is promotion; Push edges with promotion piece to Edge Arena dividing prior among edges equally
        Else Push edge without promotion piece
Else add Edge
```

4.6.3 PUCT and edge selection:

the custom Q = W − L − 0.05D, and the PUCT formula with exploration constant. Include the select_best_edge function or a snippet.

4.6.4 Mask application and renormalisation:

During node expansion, the same mask used to zero out logits in masked configurations is reused (or used for the first time in unmasked configurations) to decide which edges to add to the node, to prevent invalid state transitions. In unmasked configurations, policies are renormalised over valid squares to maintain mathematical integrity in PUCT calculations.

```
if !config.masked {
    policy.iter_mut().for_each(|e| e.1 = e.1 / (1.0 - rate) as f32);
}
```

where rate is total prior in invalid squares.

In masked configurations, logits are set to −1e9.

```
if let Some(masks) = masks {
    let mask_data = TensorData::new(masks, [batch_size, 64]);
    let mask = Tensor::<B, 2, Bool>::from_data(mask_data, device);
    let mask = mask.bool_not();

    policies = policies.clone().mask_fill(mask, -1e9);
}
```

in unmasked configurations, output distribution is renormalised over valid squares.

```
let rate: f64 = mask.iter().zip(policy.iter()).map(|(legal, policy)| if !legal
{ policy.1 as f64 } else { 0.0 }).sum();
if !config.masked {
    policy.iter_mut().for_each(|e| e.1 = e.1 / (1.0 - rate) as f32);
}
```

4.6.5 PUCT and edge selection

Edge Selection

Function select_best_edge(node_idx):
  node → self.node_arena[node_idx]
  if node.child_edge_range is None **then**

EDGE SELECTION

```
    return None
end
(start, end) → node.child_edge_range
N → node.visits
max_score → −∞
best_edge → None
for each edge_idx from start to end:
    edge → self.edge_arena[edge_idx]
    (w, d, l) → edge.mean_value
    exploitation → w − l − 0.05 · d
    exploration → edge.confidence · config.c_puct · frac(sqrt(N) + 10^(-8), 1
    + edge.visits)
    total_score → exploitation + exploration
    if total_score > max_score then
        max_score → total_score
        best_edge → edge_idx
    end
end
return best_edge
```

4.6.6 Parallel search:

Self-play throughput is maximized by parallelizing MCTS rollouts across the batch size. Using the `rayon` crate, the system executes tree traversals concurrently (`mctss.par_iter_mut()`). Because the Arena Allocator isolates each MCTS instance in contiguous memory, data races are structurally impossible, and cache invalidation is minimized.

4.7 Neural Network and Tensor Formulation

4.7.1 State canonicalisation:

The board is canonicalized to the perspective of the side-to-move when making tensors. This halves the required state-space exploration for the network.

```
NetworkInputs::from_position(position: &ChessPosition, selected_sq:
Option<&ChessSquare>) -> Self {
    let (chess_board, castling_rights, ep_sq) = if position.side_to_move ==
Color::White {
        (position.chessboard.clone(), position.castling_rights,
position.en_passant)
    } else {
        (position.chessboard.flip_board(),
position.castling_rights.flip_perspective(), position.en_passant.map(|x|
x.square_opposite()))
    };
```

```
    ...
}
```

4.7.2 Input tensor layout:

```
pub fn inputs_to_tensor<B: Backend>(buffer: &Vec<NetworkInputs>, device:
&B::Device) -> (Tensor<B, 3>, Tensor<B, 2>) {
    let n = buffer.len();

    let mut boards = Vec::with_capacity(n * 64 * 14);
    let mut metas = Vec::with_capacity(n * 5);

    for item in buffer {
        boards.extend_from_slice(&item.boards);
        metas.extend_from_slice(&item.meta);
    }

    let shape = [n, 64, 14];
    let board_data = TensorData::new(boards, shape);
    let t1 = Tensor::from_data(board_data, device);

    let shape = [n, 5];
    let meta_data = TensorData::new(metas, shape);
    let t2 = Tensor::from_data(meta_data, device);

    (t1, t2)
}
```

The spatial tensor $X \in \mathbb{R}^{64\times 14}$ reserves the 14th plane for the selected piece. During a `PieceSelect` node evaluation, this plane is strictly zeroed.

4.7.3 Two-pass forward:

The network executes two forward passes using shared weights. The policy head outputs $P(\text{from}|s)$. During a `PieceMove` node evaluation, the index of the selected square is set to 1.0 in the 14th plane. The network re-evaluates the state, outputting $P(\text{to}|s, \text{from})$. This mechanism forces the attention layers to dynamically shift focus from global piece evaluation to localized kinematic projection based solely on the presence of the 14th plane.

4.7.4 Transformer architecture:

list the layers (piece_encoder, meta_encoder, positional embedding, transformer encoder, policy/value heads). Include the forward method that concatenates the meta-token as a [CLS] token. Show the 2D positional encoding code.

```
// 2D Positional Encoding
let x_emb =
self.pos_embedding_x.forward(self.coordinates.clone()).unsqueeze_dim(1);
let y_emb =
self.pos_embedding_y.forward(self.coordinates.clone()).unsqueeze_dim(2);
let pos_flat = (x_emb + y_emb).reshape([1, 64, self.d_model]);
x = x + pos_flat;

// Meta-token concatenation (Pseudo-[CLS] token)
let meta_x = self.meta_encoder.forward(meta).unsqueeze_dim(1);
let x = Tensor::cat(vec![x, meta_x], 1);
```

4.7.5 Output processing:

the policy head outputs 64 logits → softmax; value head outputs 3 logits → softmax. Mention the model_make_outputs utility that optionally masks logits and returns NetworkLabels.

4.8 Training Pipeline

4.8.1 Self-play loop (play()):

give a high-level walkthrough of the outer loop. Use a flowchart or numbered steps: initialise MCTS and games → for each step in iteration → traverse MCTS → expand batch using model → select best move → generate training sample → push to replay buffer → every steps_per_iter iterations, perform gradient steps.

4.8.2 Expand batch (expand_batch):

explain how masks are gathered, network inference is batched, edges are created, and Dirichlet noise is added at the root. Show the core code that creates edges (including promotion splitting) and updates the node's child_edge_range.

4.8.3 Replay buffer:

describe the FIFO buffer, its size (524k samples), and the ChessBatcher that constructs tensors from sampled TrainingSamples.

The `ChessBatcher` constructs the input tensors dynamically given a slice of training samples randomly sampled from the replay buffer. The spatial tensor $X \in \mathbb{R}^{B\times 64\times 14}$ is populated by writing to flat `Vec<f32>` before being turned into tensors and reshaped.

Generated trajectories are pushed to a First-In-First-Out (FIFO) `ReplayBuffer` with a capacity of 524,288 samples. With a batch size of 256, the buffer saturates after approximately 2,048 moves. Assuming an average game length of 96 plies, the buffer contains data from the most recent $s\,\mathrm{im}\,5,400$ games. This capacity was explicitly chosen to balance data freshness (preventing the network from overfitting to outdated policies) with sufficient historical context to stabilize the gradients.

4.8.4 Self-annealing loss:

In unmasked configurations, the network must learn environmental constraints via gradient descent. This is achieved through a dynamic, self-annealing loss function. Let λ represent the average probability mass assigned to illegal moves across a batch. The loss is defined as: $L = L_{\{\text{policy}\}}(1+\lambda) + L_{\text{value}}(1-\lambda)$

When the network acts randomly ($\lambda \approx 1$), the value loss is suppressed, forcing the optimizer to prioritize kinematic legality. As the policy converges on $A_{L(s)}$ ($\lambda \approx 0$), the weights balance, allowing strategic value evaluation to propagate. This acts as an automated curriculum, implemented directly within the `ChessTransformer::calculate_loss` method.

4.8.5 Optimiser and LR schedule:

briefly state AdamW with Noam scheduler (warmup 4000 steps) and weight decay. You can reference the configuration struct.

4.9 Performance and Correctness

4.9.1 Correctness:

mention unit tests for move generation, perft-like validation (i didnt do this). describe the test strategy (e.g., comparing move counts to known perft numbers for a few FENs) than claim full verification.

4.9.2 Profiling and throughput:

report the number of MCTS simulations per second on your hardware, the average inference batch throughput, and memory usage per MCTS instance. network inference is primary bottleneck

4.9.3 Scalability:

mention that the arena-based design allowed near-linear scaling with the number of parallel games.

4.10 Summary

A short paragraph recapping the key implementation contributions: the bipartite MCTS, the two-pass network with canonicalised input, the self-annealing loss, and the efficient Rust engine.

Chapter 5

Results and Evaluation

5.1 Experimental Setup

To ensure fairness, all variants were initialized with identical seeds, trained for the same number of epochs, and evaluated at identical MCTS search depths.

5.2 Ruleset Convergence Analysis

5.3 Masking and Grokking Analysis

Chapter 6

Conclusion

6.1 Project Evaluation

6.2 Limitations and Future Work

- **Compute Constraints:** Hardware limitations restricted the total number of training epochs, preventing observation of deep late-stage grokking.
- **Move Generation:** Future iterations should replace standard bitboard raycasting with Magic Bitboards for optimal

performance.

- **Endgame Tablebases (EGTB):** Injecting perfect knowledge at varying stages of the training pipeline remains a highly viable area for future research to bootstrap the value head and sharpen endgame play.
- **Engine Evaluations** Injecting perfect knowledge from high depth engine evaluations with forced wins similarly are viable for perject knowledge injection.
- **Kinematic Bootstrapping:** Boot strap the policy head with uniform distributions of valid moves, before playing for value.

Chapter 7

Statement of Ethics

Bibliography

Bengio, Y. *et al.* (2009) "Curriculum learning," in *Proceedings of the 26th Annual International Conference on Machine Learning*. Montreal, Quebec, Canada: Association for Computing Machinery (ICML '09), pp. 41–48. Available at: https://doi.org/10.1145/1553374.1553380.

Huang, S. and Ontañón, S. (2020) "A Closer Look at Invalid Action Masking in Policy Gradient Algorithms," *CoRR* [Preprint]. Available at: https://arxiv.org/abs/2006.14171.

Jenner, E. *et al.* (2024) *Evidence of Learned Look-Ahead in a Chess-Playing Neural Network*. Available at: https://arxiv.org/abs/2406.00877.

Monroe, D. and Chalmers, P.A. (2024) *Mastering Chess with a Transformer Model*. Available at: https://arxiv.org/abs/2409.12272.

Power, A. *et al.* (2022) "Grokking: Generalization beyond overfitting on small algorithmic datasets," *arXiv preprint arXiv:2201.02177* [Preprint].

Schrittwieser, J. *et al.* (2020) "Mastering Atari, Go, chess and shogi by planning with a learned model," *Nature*, 588(7839), pp. 604–609. Available at: https://doi.org/10.1038/s41586-020-03051-4.

Shannon, C.E. (1950) "XXII. Programming a computer for playing chess," *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314), pp. 256–275. Available at: https://doi.org/10.1080/14786445008521796.

Sharma, S. *et al.* (2017) "Learning to Factor Policies and Action-Value Functions: Factored Action Space Representations for Deep Reinforcement learning," *CoRR* [Preprint]. Available at: http://arxiv.org/abs/1705.07269.

Silver, D. *et al.* (2017) *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. Available at: https://arxiv.org/abs/1712.01815.

World Chess Federation (FIDE) (2023) "FIDE Laws of Chess". Available at: https://handbook.fide.com/chapter/E012023.